

## Week-3 Assignment

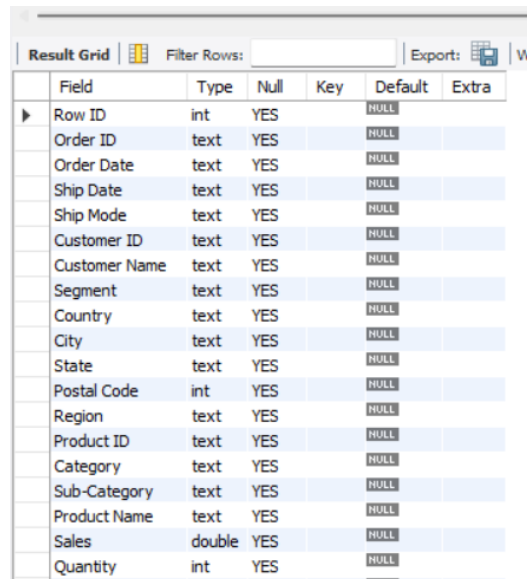
### Objective:

Use Subqueries, CTEs, and Window Functions to analyze sales data from the Superstore dataset.

### Tasks

#### Step 1: Setup Data

1. Import the Superstore dataset into a table named **superstore\_raw**.
  - Created a New Database named superstore\_db.
  - In Schema -> Right click on superstore\_db -> Select Table Data Import Wizard -> Select File path -> name the table as superstore\_raw -> Click on Import



The screenshot shows the 'Table Data Import Wizard' window. At the top, there are tabs for 'Result Grid', 'Filter Rows', and 'Export'. Below these is a table listing the columns of the dataset. The table has columns: Field, Type, Null, Key, Default, and Extra. The rows list various fields from the Superstore dataset, including Row ID, Order ID, Order Date, Ship Date, Ship Mode, Customer ID, Customer Name, Segment, Country, City, State, Postal Code, Region, Product ID, Category, Sub-Category, Product Name, Sales, and Quantity.

| Field         | Type   | Null | Key | Default | Extra |
|---------------|--------|------|-----|---------|-------|
| Row ID        | int    | YES  |     | NULL    |       |
| Order ID      | text   | YES  |     | NULL    |       |
| Order Date    | text   | YES  |     | NULL    |       |
| Ship Date     | text   | YES  |     | NULL    |       |
| Ship Mode     | text   | YES  |     | NULL    |       |
| Customer ID   | text   | YES  |     | NULL    |       |
| Customer Name | text   | YES  |     | NULL    |       |
| Segment       | text   | YES  |     | NULL    |       |
| Country       | text   | YES  |     | NULL    |       |
| City          | text   | YES  |     | NULL    |       |
| State         | text   | YES  |     | NULL    |       |
| Postal Code   | int    | YES  |     | NULL    |       |
| Region        | text   | YES  |     | NULL    |       |
| Product ID    | text   | YES  |     | NULL    |       |
| Category      | text   | YES  |     | NULL    |       |
| Sub-Category  | text   | YES  |     | NULL    |       |
| Product Name  | text   | YES  |     | NULL    |       |
| Sales         | double | YES  |     | NULL    |       |
| Quantity      | int    | YES  |     | NULL    |       |

2. Create these 3 tables from it:

#### a. customers

#CREATE TABLE table\_name:- Through this query we have created a table named customers with columns customer\_id and customer\_name.

```
CREATE TABLE customers (  
    customer_id VARCHAR(50),  
    customer_name VARCHAR(100)  
);
```

#### b. orders

#Through this query we have created a table named products which contains product info like product\_id, product\_name, category.

```
CREATE TABLE products (  
    product_id VARCHAR(50),  
    product_name VARCHAR(255),  
    category VARCHAR(100)  
);
```

### c. products

#Through this query we have created a table named orders which contains order info like order\_id, customer\_id, product\_id, order\_date, sales.

```
CREATE TABLE orders (  
    order_id VARCHAR(30),  
    customer_id VARCHAR(20),  
    product_id VARCHAR(50),  
    order_date DATE,  
    sales DECIMAL(10,2)  
);
```

## 3. Insert data into these tables using SELECT DISTINCT

### customers

#Through DISTINCT keyword we have inserted unique customer records from the raw dataset into the customers table.

```
INSERT INTO customers (customer_id, customer_name)  
  
SELECT DISTINCT  
  
    `Customer ID`,  
  
    `Customer Name`  
  
FROM superstore_raw;
```

### Products

#Through DISTINCT keyword we have inserted unique product records from the raw dataset into the products table.

```

INSERT INTO products

SELECT DISTINCT

`Product ID`,

`Product Name`,

Category

FROM superstore_raw;

```

## orders

#Through DISTINCT keyword we have inserted unique order records from the raw dataset into the orders table.

```

INSERT INTO orders

SELECT DISTINCT

`Order ID`,

`Customer ID`,

`Product ID`,

STR_TO_DATE(`Order Date`, '%m/%d/%Y'),

Sales

FROM superstore_raw;

```

## Step 2: Perform Required Queries

Write and execute SQL queries for each of the following:

1. Find all orders where sales are greater than the average sales. (Subquery)

```

66 #Subquery
67 #Used to find orders whose sales value is greater than the average sales.
68 • SELECT * FROM orders
69 WHERE Sales > (SELECT AVG(Sales) FROM orders);

```

| order_id       | customer_id | product_id      | order_date | sales   |
|----------------|-------------|-----------------|------------|---------|
| CA-2016-152156 | CG-12520    | FUR-BO-10001798 | 2016-11-08 | 261.96  |
| CA-2016-152156 | CG-12520    | FUR-CH-10000454 | 2016-11-08 | 731.94  |
| US-2015-108966 | SO-20335    | FUR-TA-10000577 | 2015-10-11 | 957.58  |
| CA-2014-115812 | BH-11710    | TEC-PH-10002275 | 2014-06-09 | 907.15  |
| CA-2014-115812 | BH-11710    | FUR-TA-10001539 | 2014-06-09 | 1706.18 |
| CA-2014-115812 | BH-11710    | TEC-PH-10002033 | 2014-06-09 | 911.42  |
| CA-2016-161389 | IM-15070    | OFF-BI-10003656 | 2016-12-05 | 407.98  |
| CA-2014-105893 | PK-19075    | OFF-ST-10004186 | 2014-11-11 | 665.88  |

2. Find the highest sales order for each customer. (Subquery)

```

72 • SELECT * FROM orders
73   WHERE sales IN (
74     SELECT MAX(sales)
75     FROM orders
76     GROUP BY customer_id
77   );

```

| order_id       | customer_id | product_id      | order_date | sales   |
|----------------|-------------|-----------------|------------|---------|
| CA-2016-152156 | CG-12520    | FUR-CH-10000454 | 2016-11-08 | 731.94  |
| US-2015-108966 | SO-20335    | FUR-TA-10000577 | 2015-10-11 | 957.58  |
| CA-2014-115812 | BH-11710    | FUR-TA-10001539 | 2014-06-09 | 1706.18 |
| CA-2015-106320 | EB-13870    | FUR-TA-10000577 | 2015-09-25 | 1044.63 |
| US-2015-150630 | TB-21520    | FUR-BO-10004834 | 2015-09-17 | 3083.43 |
| CA-2016-117590 | GH-14485    | TEC-PH-10004977 | 2016-12-08 | 1097.54 |
| CA-2015-117415 | SN-20710    | FUR-BO-10002545 | 2015-12-27 | 532.40  |
| CA-2016-105816 | JM-15265    | TEC-PH-10002447 | 2016-12-11 | 1029.95 |
| CA-2016-111682 | TB-21055    | FUR-CH-10003968 | 2016-06-17 | 319.41  |
| CA-2014-106376 | BS-11590    | OFF-AR-10002671 | 2014-12-05 | 1113.02 |
| CA-2017-140088 | PO-18865    | FUR-CH-10000863 | 2017-05-28 | 301.96  |
| CA-2015-129476 | PA-19060    | TEC-AC-10000844 | 2015-10-15 | 339.96  |
| US-2014-152030 | AD-10180    | FUR-CH-10004063 | 2014-12-26 | 600.56  |
| CA-2015-110744 | HA-14920    | OFF-ST-10003656 | 2015-09-07 | 671.93  |
| CA-2016-114489 | JE-16165    | FUR-CH-10000454 | 2016-12-05 | 1951.84 |
| CA-2014-139892 | BM-11140    | TEC-MA-10000822 | 2014-09-08 | 8159.95 |

### 3. Calculate total sales for each customer. (CTE)

```

77 #CTE
78 #Used to calculate total sales generated by each customer using a CTE
79 • WITH customer_sales AS
80   (
81     SELECT customer_id,
82            SUM(Sales) AS TotalSales
83     FROM orders
84     GROUP BY customer_id
85   )
86   SELECT *
87   FROM customer_sales;

```

| customer_id | TotalSales |
|-------------|------------|
| CG-12520    | 1148.78    |
| DV-13045    | 1119.48    |
| SO-20335    | 2602.58    |
| BH-11710    | 6255.34    |
| AA-10480    | 1782.87    |
| IM-15070    | 4930.49    |
| HP-14815    | 1990.31    |
| PK-19075    | 8158.65    |
| AG-10270    | 2565.26    |
| ZD-21925    | 1493.94    |
| KB-16585    | 2742.76    |
| SF-20065    | 763.54     |
| EB-13870    | 2767.22    |
| EH-13945    | 684.18     |
| TB-21520    | 4730.62    |

### 4. Find customers whose total sales are above average. (CTE + Subquery)

```

89  #CTE + Subquery
90  #Used to display customers whose total sales are above the average customer sales
91  WITH customer_sales AS(
92  SELECT customer_id,
93  SUM(Sales) AS TotalSales
94  FROM orders
95  GROUP BY customer_id
96  )
97  SELECT *
98  FROM customer_sales
99  WHERE TotalSales > (
100 SELECT AVG(TotalSales)
101 FROM customer_sales
102 );

```

| customer_id | TotalSales |
|-------------|------------|
| BH-11710    | 6255.34    |
| IM-15070    | 4930.49    |
| PK-19075    | 8158.65    |
| TB-21520    | 4730.62    |
| MA-17560    | 4280.62    |
| SN-20710    | 3127.97    |
| LC-16930    | 4481.16    |
| RA-19885    | 3832.32    |
| ES-14080    | 4648.52    |
| KM-16720    | 4909.47    |
| KD-16270    | 8282.36    |

5. Rank all customers based on total sales. (Window Function)

```

104  #Window Function
105  #Used to rank customers based on their total sales in descending order
106  SELECT customer_id,
107  SUM(Sales) AS TotalSales,
108  RANK() OVER(
109  ORDER BY SUM(Sales) DESC
110  ) AS RankNo
111  FROM orders
112  GROUP BY customer_id;

```

| customer_id | TotalSales | RankNo |
|-------------|------------|--------|
| SM-20320    | 25043.07   | 1      |
| TC-20980    | 19017.85   | 2      |
| RB-19360    | 15117.35   | 3      |
| TA-21385    | 14595.62   | 4      |
| AB-10105    | 14355.61   | 5      |
| SC-20095    | 14142.34   | 6      |
| KL-16645    | 14071.92   | 7      |
| HL-15040    | 12873.30   | 8      |
| SE-20110    | 12209.44   | 9      |
| CC-12370    | 12129.08   | 10     |
| TS-21370    | 11885.87   | 11     |
| GT-14710    | 11820.12   | 12     |
| BM-11140    | 11609.91   | 13     |

6. Assign row numbers to each order within a customer. (Window Function + PARTITION BY)

```

114 #Window Function + PARTITION BY
115 #Used to assign a row number to each order within every customer group
116 • SELECT customer_id,
117         order_id,
118         Sales,
119         ROW_NUMBER() OVER(
120     PARTITION BY customer_id
121     ORDER BY Sales DESC
122 ) AS RowNum
123 FROM orders;

```

| customer_id | order_id       | Sales   | RowNum |
|-------------|----------------|---------|--------|
| AA-10315    | CA-2016-103982 | 3930.07 | 1      |
| AA-10315    | CA-2014-128055 | 673.57  | 2      |
| AA-10315    | CA-2016-103982 | 431.98  | 3      |
| AA-10315    | CA-2017-147039 | 362.94  | 4      |
| AA-10315    | CA-2014-128055 | 52.98   | 5      |
| AA-10315    | CA-2016-103982 | 41.72   | 6      |
| AA-10315    | CA-2015-121391 | 26.96   | 7      |
| AA-10315    | CA-2014-138100 | 14.94   | 8      |
| AA-10315    | CA-2014-138100 | 14.56   | 9      |
| AA-10315    | CA-2017-147039 | 11.54   | 10     |
| AA-10315    | CA-2016-103982 | 2.30    | 11     |
| AA-10375    | CA-2016-131065 | 499.98  | 1      |
| AA-10375    | CA-2015-140921 | 149.97  | 2      |
| AA-10375    | CA-2017-100230 | 116.98  | 3      |

## 7. Display top 3 customers based on total sales. (Window Function)

```

125 #Window Function
126 # Used to retrieve the top 3 customers based on total sales.
127 • WITH customer_sales AS(
128     SELECT customer_id,
129           SUM(Sales) AS TotalSales,
130           RANK() OVER(
131       ORDER BY SUM(Sales) DESC
132     ) AS RankNo FROM orders
133     GROUP BY customer_id
134 )
135 SELECT * FROM customer_sales
136 WHERE RankNo <= 3;

```

| customer_id | TotalSales | RankNo |
|-------------|------------|--------|
| SM-20320    | 25043.07   | 1      |
| TC-20980    | 19017.85   | 2      |
| RB-19360    | 15117.35   | 3      |

## Step 3: Final Combined Query

Write **one final query** that shows:

- Customer Name
- Total Sales
- Rank

(Use JOIN + CTE + Window Function together)

```
138 #Final Combined Query
139 #JOIN + CTE + Window Function
140 #Used to display customer ID, total sales, and sales rank using CTE, JOIN, and Window Function.
141 WITH customer_sales AS(
142     SELECT customer_id,
143     SUM(Sales) AS TotalSales
144     FROM orders
145     GROUP BY customer_id
146 )
147 SELECT c.customer_id,
148 cs.TotalSales,
149 RANK() OVER(
150     ORDER BY cs.TotalSales DESC
151 ) AS RankNo
152 FROM customer_sales cs
153 JOIN customers c
154 ON cs.customer_id = c.customer_id;
```

| customer_id | TotalSales | RankNo |
|-------------|------------|--------|
| SM-20320    | 25043.07   | 1      |
| SM-20320    | 25043.07   | 1      |
| SM-20320    | 25043.07   | 1      |
| TC-20980    | 19017.85   | 4      |
| TC-20980    | 19017.85   | 4      |
| TC-20980    | 19017.85   | 4      |
| RB-19360    | 15117.35   | 7      |
| RB-19360    | 15117.35   | 7      |
| RB-19360    | 15117.35   | 7      |

## Mini Project: Customer Sales Insights

Answer the following using SQL:

1. Who are the top 5 customers?

```
156 #Mini Project: Customer Sales Insights
157 #Used to find the top 5 customers based on total sales.
158 • SELECT `Customer Name`,
159 SUM(Sales) AS TotalSales
160 FROM superstore_raw
161 GROUP BY `Customer Name`
162 ORDER BY TotalSales DESC
163 LIMIT 5;
```

| Customer Name | TotalSales         |
|---------------|--------------------|
| Sean Miller   | 25043.05           |
| Tamara Chand  | 19017.847999999998 |
| Raymond Buch  | 15117.339          |
| Tom Ashbrook  | 14595.62           |
| Adrian Barton | 14355.610999999997 |

2. Who are the bottom 5 customers?

```
165 #Used to find the bottom 5 customers based on total sales.
166 • SELECT `Customer Name`,
167 SUM(Sales) AS TotalSales
168 FROM superstore_raw
169 GROUP BY `Customer Name`
170 ORDER BY TotalSales ASC
171 LIMIT 5;
172
```

| Customer Name   | TotalSales |
|-----------------|------------|
| Thais Sissman   | 4.833      |
| Lela Donovan    | 5.304      |
| Mitch Gastineau | 12.32      |
| Carl Jackson    | 16.52      |
| Roy Skaria      | 22.328     |

3. Which customers made only one order?

```
173 #Used to identify customers who have placed only one order.
174 • SELECT `Customer Name`
175 FROM superstore_raw
176 GROUP BY `Customer Name`
177 HAVING COUNT(DISTINCT `Order ID`) = 1;
178
```

| Customer Name     |
|-------------------|
| Anemone Ratner    |
| Anthony O'Donnell |
| Carl Jackson      |
| Jenna Caffey      |
| Jocasta Rupert    |
| Lela Donovan      |
| Mitch Gastineau   |
| Patricia Hirasaki |
| Ricardo Emerson   |
| Roland Murray     |
| Susan Mackendrick |
| Theresa Coyne     |



#### 4. Which customers have above-average sales?

```
179 #Used to find customers whose total sales are above the overall average sales.
180 • SELECT `Customer Name`,
181       SUM(Sales) AS TotalSales
182 FROM superstore_raw
183 GROUP BY `Customer Name`
184 HAVING SUM(Sales) > (
185   SELECT AVG(Sales)
186   FROM superstore_raw
187 );
```

| Customer Name      | TotalSales         |
|--------------------|--------------------|
| Claire Gute        | 1148.7800000000002 |
| Darrin Van Huff    | 1119.4830000000002 |
| Sean O'Donnell     | 2602.5750000000001 |
| Brosina Hoffman    | 6255.3510000000001 |
| Andrew Allen       | 1782.872           |
| Irene Maddox       | 4930.4739999999999 |
| Harold Pawlan      | 1990.314           |
| Pete Kriz          | 8158.654           |
| Alejandro Grove    | 2565.2579999999994 |
| Zuschuss Donatelli | 1493.944           |
| Ken Black          | 2742.7580000000003 |
| Sandra Flanagan    | 763.5459999999998  |
| Emily Burns        | 2767.219           |
| Eric Hoffmann      | 684.171            |

#### 5. What is the highest order value per customer?

```
189 #Used to display the highest order value recorded for each customer.
190 • SELECT `Customer Name`,
191       MAX(Sales) AS HighestOrder
192 FROM superstore_raw
193 GROUP BY `Customer Name`;
```

| Customer Name      | HighestOrder |
|--------------------|--------------|
| Claire Gute        | 731.94       |
| Darrin Van Huff    | 721.875      |
| Sean O'Donnell     | 957.5775     |
| Brosina Hoffman    | 1706.184     |
| Andrew Allen       | 479.97       |
| Irene Maddox       | 1819.86      |
| Harold Pawlan      | 1242.9       |
| Pete Kriz          | 4305.552     |
| Alejandro Grove    | 1336.44      |
| Zuschuss Donatelli | 823.96       |
| Ken Black          | 1013.832     |
| Sandra Flanagan    | 254.9        |
| Emily Burns        | 1044.63      |
| Eric Hoffmann      | 100.704      |